Clause-Guard

Automated Lease & Rental Agreement Analyzer

Project Documentation

Project Title	Clause-Guard: Automated Lease & Rental Agreement Analyzer
Student Name(s)	[Student Name]
Course / Subject	Software Engineering
Instructor Name	[Instructor Name]
Date	28 March 2026

1. Objective of Project

1.1 Purpose of the Project

Clause-Guard is an AI-powered legal technology tool designed to bridge the information gap between landlords and tenants. By leveraging Natural Language Processing (NLP) and Retrieval-Augmented Generation (RAG), the system analyses rental agreements uploaded by users, identifies predatory or illegal clauses based on specific jurisdictional laws, and translates complex legal jargon into plain, understandable English.

1.2 Problem It Addresses

Tenants frequently sign exploitative lease agreements because they lack legal knowledge or cannot afford legal counsel. Lease documents are dense with legalese, making it difficult for ordinary individuals to identify clauses that may be unfair or outright illegal under local tenancy laws.

This information asymmetry creates a power imbalance between landlords and tenants, often resulting in financial harm or loss of tenant rights.

1.3 Expected Outcomes

- A web-based platform that parses PDF lease/rental agreements uploaded by users.
- Automatic jurisdiction-aware analysis of lease clauses against a legal knowledge base.
- Classification of every clause as Fair, Unfair (predatory but legal), or Illegal (violates local statutes).
- An aggregate Risk Score (0–100) summarising the overall risk level of the document.
- Side-by-side display of original legalese versus a plain-English explanation for each flagged clause.
- A downloadable PDF summary report of the analysis.
- Optional user account to save and track analysis history across sessions.

2. Requirements Specification

This section provides a comprehensive overview of the system requirements for Clause-Guard, covering both what the system must do (functional) and the quality standards it must meet (non-functional).

2.1 Functional Requirements

Module	Requirement	Justification
User Module	Anonymous PDF upload (Guest access)	Lowers the barrier to entry — users can analyse a lease without creating an account, maximising accessibility.
User Module	User registration and login with email verification	Enables persistent report history and personalised experience for returning users.
Parsing Module	Accept PDF files up to 10 MB	Covers the size of typical lease agreements while preventing server overload from excessively large files.
Parsing Module	OCR fallback using Tesseract for scanned PDFs	Many leases are scanned paper documents with no text layer; OCR ensures the system can handle real-world uploads.
Analysis Module	Jurisdiction selection (country + state/region)	Legal standards differ by location; jurisdiction filtering ensures the correct laws are retrieved from the vector database.
Analysis Module	Regex-based structured data extraction (rent, dates, deposit)	Provides fast, precise extraction of key financial and temporal terms, supplementing semantic analysis.
Analysis Module	RAG-based semantic clause analysis using ChromaDB + LLM	Enables nuanced understanding of clause intent beyond keyword matching, reducing false positives.
Analysis Module	Clause classification: Fair / Unfair / Illegal	Gives the user a clear, actionable verdict for each clause with an associated severity weight.
Report Module	Aggregate Risk Score (0–100) calculation	Condenses complex analysis into a single, intuitive score (Low / Medium / High) for quick decision-making.
Report Module	Side-by-side original legalese vs plain-English explanation	Makes the analysis accessible to users without legal expertise.
Report Module	Export analysis as PDF report	Registered users can download and share the report or present it to a lawyer or landlord.
Admin Module	Admin panel to manage legal knowledge base in ChromaDB	Keeps jurisdiction-specific statute data current and accurate, directly impacting analysis quality.

2.2 Non-Functional Requirements

Quality Attribute	Requirement	Justification
Performance	Full analysis completes in under 15 seconds	Users expect near-real-time feedback; long waits reduce trust in the system.
Accuracy	Rule-based extraction >95%; semantic classification >85%	High accuracy is critical for a tool used to make legally significant decisions.
Privacy & Security	Uploaded PDFs deleted immediately after analysis (unless saved by user)	Lease agreements contain sensitive personal and financial information.
Security	Passwords hashed with bcrypt; account lockout after 5 failed attempts	Protects user accounts from brute-force attacks.
Scalability	New jurisdiction law-sets can be added without codebase changes	Future expansion to new regions must not require re-engineering the system.
Usability	User-friendly interface accessible without technical knowledge	Target users are everyday tenants, not technical professionals.
Reliability	Rule-based fallback if LLM API is unavailable	System must continue to provide useful output even when the external AI service is down.

3. Use Case Diagram

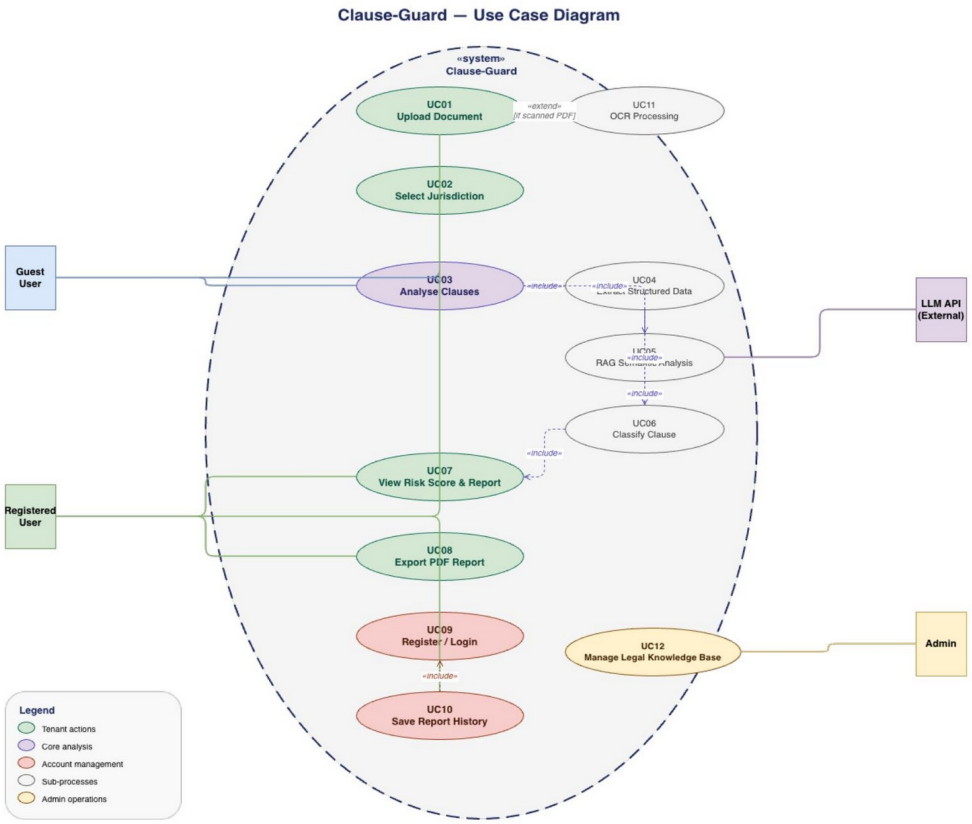

Figure 1 — Clause-Guard Use Case Diagram

3.1 Actors

The following actors interact with the Clause-Guard system:

Actor	Description
Guest User	An unauthenticated visitor who can upload documents, select a jurisdiction, trigger analysis, and view the generated report. Results are stored in temporary session memory and are lost when the browser closes.
Registered User	An authenticated tenant with all Guest User capabilities, plus the ability to save reports, view report history, and export PDF reports. Must complete email registration and verification.
Admin	A privileged internal user responsible for maintaining the legal knowledge base in ChromaDB. Adds new jurisdiction statutes, updates outdated laws, and ensures vector database accuracy.
LLM API	An external AI service (GPT-4 / Claude) that receives structured prompts containing a lease clause and retrieved statutes, and returns a compliance classification, plain-English explanation, and cited statute reference.

3.2 Use Cases and Interactions

The system boundary contains twelve use cases grouped by category:

ID	Use Case	Actor(s)	Relationship
UC01	Upload Document	Guest, Registered User	«extend» UC11 — OCR Processing (if scanned PDF)
UC02	Select Jurisdiction	Guest, Registered User	Precedes UC03
UC03	Analyse Clauses	Guest, Registered User, LLM API	«include» UC04, UC05, UC06
UC04	Extract Structured Data	System (internal)	«include» by UC03
UC05	RAG Semantic Analysis	System, LLM API	«include» by UC03; «include» UC06
UC06	Classify Clause	System (internal)	«include» by UC03/UC04/UC05; «include» UC07
UC07	View Risk Score & Report	Guest, Registered User	«include» by UC06
UC08	Export PDF Report	Registered User	Requires UC07 precondition
UC09	Register / Login	Registered User	«include» by UC10
UC10	Save Report History	Registered User	«include» UC09
UC11	OCR Processing	System (internal)	«extend» UC01 (only for scanned PDFs)
UC12	Manage Legal	Admin	Independent; updates ChromaDB

ID	Use Case	Actor(s)	Relationship
	Knowledge Base		

4. Activity Diagram

4.1 Complete Analysis Workflow

The activity diagram below models the end-to-end business process of the Clause-Guard system, from a user uploading a document through to viewing the final risk report.

Activity Diagram — Complete Document Analysis Workflow

Workflow Steps:

1. Start: User uploads a PDF lease agreement.
2. File Validation: Browser validates file type (PDF) and size (≤ 10 MB). If invalid → show error and end.
3. Jurisdiction Selection: User selects country and state. If unsupported → notify user with option for generic analysis.
4. Text Extraction (Decision Fork): - If text-based PDF → extract using pdfplumber / PyPDF2. - If scanned PDF (no text layer) → invoke Tesseract OCR pipeline.
5. Clause Segmentation: Extracted text is cleaned and split into individual clauses.
6. Parallel Processing (Fork): - Branch A: Regex extraction of structured data (rent amounts, dates, deposit values). - Branch B: Generate 384-dim embeddings; query ChromaDB for top 3–5 matching statutes.
7. LLM Analysis Loop (For each clause): - Construct prompt from clause + retrieved statutes. - Call LLM API (GPT-4 / Claude). - If API unavailable → retry up to 3 times, then use rule-based fallback. - Parse response: classification (Fair / Unfair / Illegal), explanation, statute cited.
8. Clause Classification: Map LLM output to severity weight (Fair=0, Unfair=10, Illegal=30).
9. Risk Score Calculation: Sum all weights → normalise to 0–100 scale → assign category (Low <30, Medium 30–59, High ≥60).
10. Report Generation: Render interactive report with score gauge, flagged clauses, and plain-English explanations.
11. Session Check (Decision): - Registered User → save report to SQLite/Firebase database. - Guest User → store in temporary session memory (expires on browser close).
12. End: Display results dashboard to user.

4.2 RAG Analysis Sub-Process

For each individual clause, the RAG (Retrieval-Augmented Generation) analysis sub-process operates as follows:

- Clause text is cleaned and normalised (headers, footers, whitespace removed).
- A 384-dimensional embedding vector is generated using the sentence-transformers model.

- ChromaDB is queried with a jurisdiction metadata filter; cosine similarity determines the top 3–5 statute matches.
- If similarity score < 0.7, generic legal principles are used in place of specific statutes.
- LLM prompt is constructed: [System instructions] + [Clause text] + [Retrieved statutes].
- LLM returns a JSON object: { classification, explanation, statute_cited }.
- Classification is mapped to a severity weight for the Risk Score accumulator.

5. Data Flow Diagram (DFD)

The Level 1 DFD below illustrates how data moves between external entities, processes, and data stores within the Clause-Guard system.

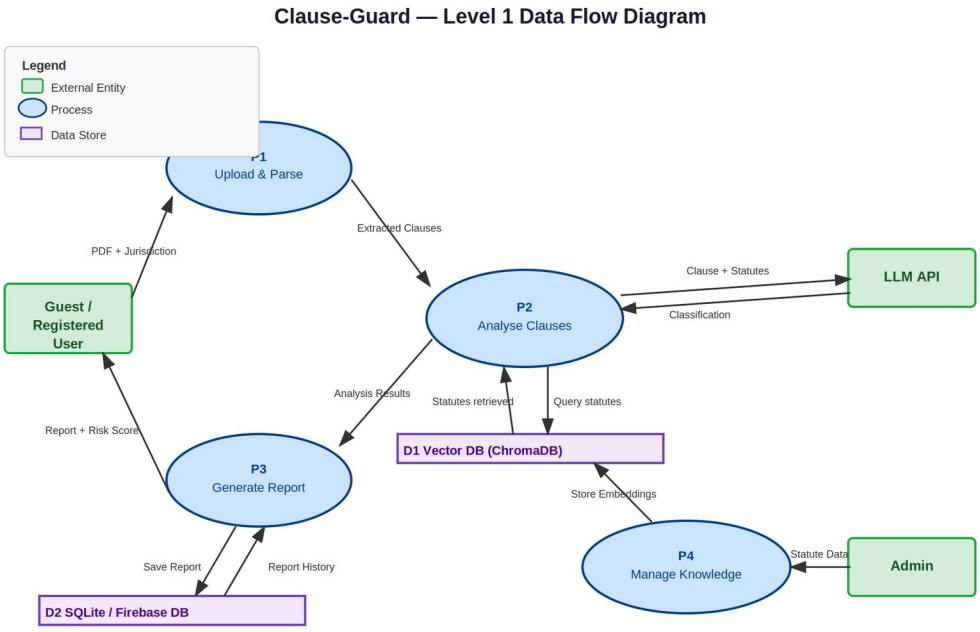

Figure 2 — Clause-Guard Level 1 Data Flow Diagram

5.1 External Entities

Entity	Description
Guest / Registered User	Initiates the system by uploading a PDF and receives the risk report in return.
Admin	Supplies statute data to the knowledge management process for storage in ChromaDB.
LLM API	Receives clause + statute prompts from the analysis process and returns classification results.

5.2 Processes

ID	Process	Data Flows
P1	Upload & Parse	In: PDF + Jurisdiction (from User). Out: Extracted Clauses (to P2).
P2	Analyse Clauses	In: Extracted Clauses (from P1); Statutes (from D1); Classification

ID	Process	Data Flows
		(from LLM API). Out: Analysis Results (to P3); Clause + Statutes (to LLM API).
P3	Generate Report	In: Analysis Results (from P2); Report History (from D2). Out: Report + Risk Score (to User); Save Report (to D2).
P4	Manage Knowledge Base	In: Statute Data (from Admin). Out: Store Embeddings (to D1).

5.3 Data Stores

ID	Store	Contents
D1	Vector DB (ChromaDB)	Jurisdiction-tagged legal statute embeddings used for semantic retrieval during clause analysis.
D2	SQLite / Firebase DB	User accounts, authentication tokens, saved reports, report metadata, and analysis history.

6. Sequence Diagram

6.1 Document Upload and Analysis — Complete Flow

The sequence diagram below details the interaction between all system components during the primary analysis workflow, showing the temporal ordering of messages and responses.

Participants: User, Web Browser (Frontend), API Server (Backend), PDF Parser, OCR Engine (Tesseract), Embedding Generator, Vector Database (ChromaDB), LLM API (GPT-4 / Claude), SQLite / Firebase Database.

Sequence of Interactions:

Step	From	To	Message / Action
1	User	Browser	Upload PDF file
2	Browser	Browser	Validate file (size ≤ 10 MB, type = PDF)
3	Browser	API Server	POST /upload (PDF + jurisdiction)
4	API Server	API Server	Create session ID; begin pipeline
5	API Server	PDF Parser	Extract text from PDF
6 [alt]	PDF Parser	PDF Parser	If text-based: extract with PyPDF2/pdfplumber
6 [else]	PDF Parser	OCR Engine	If scanned: convert pages to images → Tesseract OCR → return text
7	PDF Parser	API Server	Return extracted text
8	API Server	API Server	Segment text into individual clauses
9 [loop]	API Server	Embedding Gen.	Generate 384-dim embedding for clause
10 [loop]	API Server	Vector DB	Query similar statutes (jurisdiction filter, top 3–5)
11 [loop]	Vector DB	API Server	Return statute matches with similarity scores
12	API Server	LLM API	POST /v1/messages (clause

Step	From	To	Message / Action
[loop]			+ statutes + instructions, max_tokens=1000)
13 [loop]	LLM API	API Server	Return {classification, explanation, statute_cited}
14 [loop]	API Server	API Server	Store clause result; assign severity weight
15	API Server	API Server	Calculate aggregate Risk Score (0–100)
16	API Server	Database	INSERT INTO reports (user_id, risk_score, clauses, created_at)
17	API Server	Browser	Return report data + Risk Score
18	Browser	User	Display interactive results dashboard

6.2 User Registration and Authentication — Sequence

This sequence covers the registration and email-verification flow for new users, and the authentication flow for returning users.

Step	From	To	Message / Action
1	User	Browser	Enter email + password
2	Browser	API Server	POST /register (email, password)
3	API Server	Database	SELECT email FROM users WHERE email = ?
4 [alt]	API Server	Browser	If email exists → return error 'Email already registered'
4 [else]	API Server	Password Hasher	Hash password using bcrypt (rounds=10)
5	API Server	Database	INSERT INTO users (email, password_hash, verified=false)
6	API Server	Email Service	Send verification email with token link
7	User	Browser	Click verification link
8	Browser	API Server	GET /verify?token=xyz
9	API Server	Database	UPDATE users SET verified=true WHERE token = ?

Step	From	To	Message / Action
10	API Server	Browser	Redirect to login page; show 'Email verified'

7. Class Diagram

The class diagram below models all major entities in the Clause-Guard system, their attributes, methods, and the relationships (inheritance, association, dependency) between them.

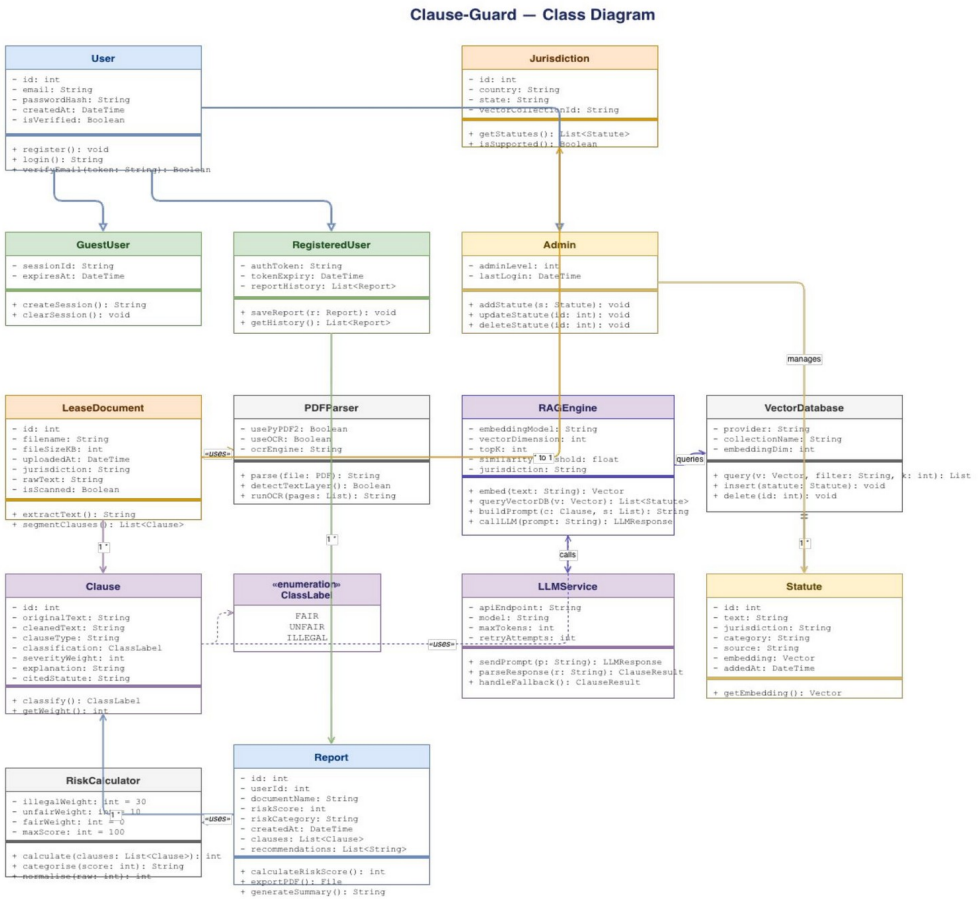

Figure 3 — Clause-Guard Class Diagram

7.1 Classes, Attributes, and Methods

Class	Key Attributes	Key Methods / Role
User (Abstract)	id, email, passwordHash, createdAt, isVerified	register(), login(), verifyEmailToken() — Base class for all user types.
GuestUser	sessionId, expiresAt	createSession(), clearSession() — Manages temporary anonymous sessions.
RegisteredUser	authToken, tokenExpiry, reportHistory	saveReport(), getHistory() — Extends User with persistence capabilities.
Admin	adminLevel, lastLogin	addStatute(), updateStatute(), deleteStatute() — Manages ChromaDB content.
Jurisdiction	id, country, state, vectorCollectionId	getStatutes(), isSupported() — Represents a geographic legal context.
LeaseDocument	filename, fileSizeKB, jurisdiction, rawText, isScanned	extractText(), segmentClauses() — Encapsulates the uploaded document.
PDFParser	usePyPDF2, useOCR, ocrEngine	parse(), detectTextLayer(), runOCR() — Handles text extraction logic.
Clause	originalText, cleanedText, clauseType, classification, severityWeight, explanation, citedStatute	classify(), getWeight() — Core domain object representing one analysed lease clause.
ClassLabel (Enum)	FAIR, UNFAIR, ILLEGAL	Enumeration of possible clause classifications.
RAGEngine	embeddingModel, vectorDimensions, topK, similarityThreshold, jurisdiction	embed(), queryVectorDB(), buildPrompt(), callLLM() — Orchestrates the RAG pipeline.
VectorDatabase	provider, collectionName, embeddingDim	query(), insert(), delete() — Abstraction over ChromaDB.
LLMService	apiEndpoint, model, maxTokens, retryAttempts	sendPrompt(), parseResponse(), handleFallback() — Manages LLM API communication.
Statute	text, jurisdiction, category, source, embedding, addedAt	getEmbedding() — Represents a single legal statute stored in ChromaDB.
RiskCalculator	illegalWeight=30, unfairWeight=10, fairWeight=0, maxScore=100	calculate(), categorise(), normalise() — Computes the aggregate Risk Score.
Report	userId, documentName, riskScore, riskCategory, clauses, recommendations	calculateRiskScore(), exportPDF(), generateSummary() — Final analysis output object.

7.2 Key Relationships

- User ◄— GuestUser / RegisteredUser / Admin: Inheritance (specialisation). All user types extend the base User class.
- LeaseDocument —uses→ PDFParser: Dependency. LeaseDocument delegates text extraction to PDFParser.
- RAGEngine —calls→ LLMService: Association. RAGEngine invokes LLMService for each clause prompt.
- RAGEngine —queries→ VectorDatabase: Association. Retrieves semantically similar statutes.
- Admin —manages→ VectorDatabase: Association. Admin adds/updates statutes in ChromaDB via the admin panel.
- Clause —uses→ ClassLabel (Enum): Dependency. Each Clause holds a ClassLabel classification value.
- RiskCalculator —uses→ Report: Dependency. Calculates and populates the Report's riskScore and riskCategory.

8. Conclusion

8.1 Summary of Project Work

Clause-Guard is a full-stack AI-powered legal technology platform that addresses a genuine societal need: the information asymmetry between landlords and tenants in residential lease agreements. The project integrates multiple advanced software engineering concepts into a cohesive, modular system:

- Natural Language Processing and Retrieval-Augmented Generation (RAG) for semantically-aware legal clause analysis.
- Vector database technology (ChromaDB) for jurisdiction-specific statute retrieval using cosine similarity search.
- OCR pipeline (Tesseract) to handle real-world scanned lease documents with no extractable text layer.
- Hybrid analysis combining rule-based regex extraction with LLM-driven semantic classification.
- A clean, layered architecture separating the Parsing, Analysis, Reporting, and User Management modules.

8.2 Key Learnings and Outcomes

The development of this project provided deep practical exposure to:

- Designing complex, multi-actor UML models including Use Case, Class, Sequence, Activity, and Data Flow Diagrams.
- Architecting a RAG system: understanding the ingestion pipeline (chunk → embed → store) and the retrieval pipeline (embed query → similarity search → prompt construction).
- The importance of non-functional requirements — particularly privacy, latency, and fallback reliability — in systems that handle sensitive personal data.
- Translating domain requirements into precise functional specifications (UC01–UC12) with clear preconditions, postconditions, and alternate flows.
- Balancing system complexity with academic feasibility, demonstrating that a well-scoped AI project can achieve both social impact and technical depth.

The Clause-Guard system, as documented in this report, is readily implementable using the specified technology stack (React.js, Python/FastAPI, ChromaDB, SQLite/Firebase, LangChain, Tesseract) and provides a strong foundation for future extensions such as a Negotiation Bot, Lease Comparison tool, and Legal Aid Marketplace integration.

9. References

Websites & Online Documentation

13. Anthropic — Claude API Documentation. https://docs.anthropic.com
14. OpenAI — GPT-4 API Documentation. https://platform.openai.com/docs
15. ChromaDB — Open-source Vector Database Documentation. https://docs.trychroma.com
16. LangChain — Framework for LLM Applications. https://python.langchain.com/docs
17. Tesseract OCR — Open Source OCR Engine. https://github.com/tesseract-ocr/tesseract
18. pdfplumber — PDF Text Extraction Library. https://github.com/jsvine/pdfplumber

Research Papers

19. Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Advances in Neural Information Processing Systems (NeurIPS), 33.
20. Reimers, N. & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. Proceedings of EMNLP 2019.

Books

21. Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education.
22. Larman, C. (2004). Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design (3rd ed.). Prentice Hall.